Benchmarking & Design Evidence

From offline design studies to on-target measurement

Overview

Two kinds of evidence back the database's design: offline Python studies that compare **candidate** designs before they're implemented, and on-target C benchmarks that measure the **shipped** implementation. This chapter documents both, including the two `pythonSim` scripts that are currently untracked in git and have no other write-up.

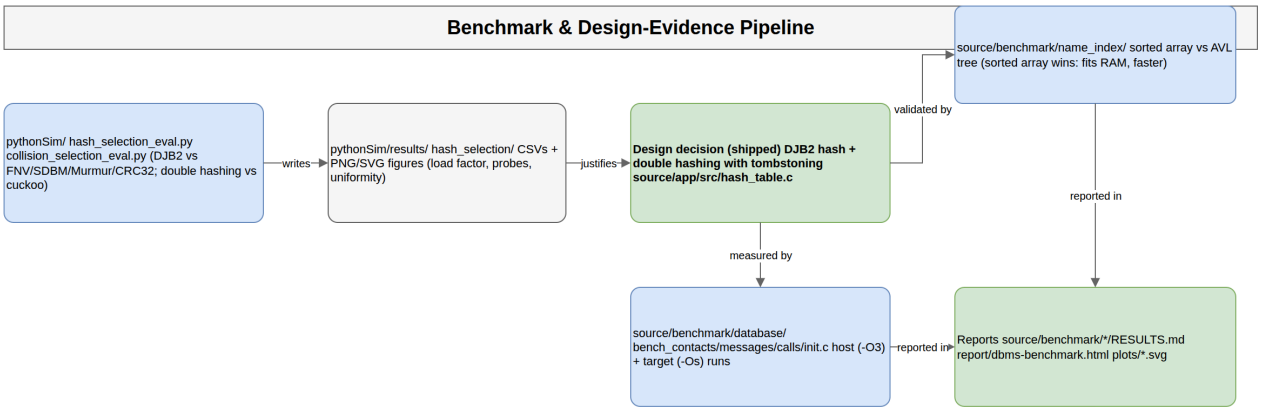


Figure 1: From offline design study to shipped code to on-target measurement.

Hashing and collision design (pythonSim)

Two scripts in `pythonSim/` produce the evidence behind the hash table's design. Both are currently **untracked** – they exist on disk but have never been committed, and this is their only documentation.

`hash_selection_eval.py`

Compares seven candidate hash functions (DJB2 16-bit – the exact model of the firmware's `hash_phone()` –, DJB2, FNV-1a 16-bit, FNV-1a, SDBM, Murmur finalizer, CRC32) against an ideal-uniform-hash reference, using paired datasets so every function sees the same phone numbers. Writes CSVs and figures to `pythonSim/results/hash_selection/`.

Headline result (100k phone numbers into a 51k-bucket table, stressed well past normal load to separate the candidates):

Hash function	Collision rate	chi²/dof	Mean probes @ 70%
DJB2 16-bit (shipped)	85.72%	1.08	1.75
FNV-1a 16-bit	85.73%	1.10	1.75
SDBM	85.72%	1.01	1.82
Murmur finalizer	85.72%	1.01	1.75
CRC32	85.72%	0.98	1.75

All seven candidates land within noise of each other on uniformity and mean probe count – chi²/dof near 1.0 means each is close to an ideal uniform hash. **No candidate meaningfully outperforms DJB2**, so the simplest option already shipped in `hash_table.c` is also the justified one.

`collision_selection_eval.py`

Explicitly "replaces `collision_eval.py` and `cuckoo.py`" with a like-for-like comparison of double hashing (the firmware's actual probe sequence, read from `hash_table.c`), linear probing, and cuckoo hashing (two independent 16-bit hashes, its most favourable form), all on the same 14,293-slot table and the same phone datasets. Measures successful and unsuccessful lookup cost, worst case, and SD reads caused by 16-bit hash collisions. Writes to `pythonSim/results/hash_selection/` with a `collision_` prefix.

Headline result (10,005 entries at 70% load, 100 simulations):

Method	Mean hit probes	Worst-case hit probes	Always fit all entries?
Linear probing	2.30	175	yes
Double hashing (shipped)	1.75	36	yes
Cuckoo hashing (2 hashes)	1.31	2	**no** – 0% of runs

Cuckoo hashing has the best **lookup** cost by far, but with only two hash functions in a single table it failed to insert every entry in **any** of the 100 simulations at 70% load (mean 7,356 / 10,005 entries actually fit) – the well-known cuckoo capacity cliff. Reaching reliable insertion would need more hash functions, multiple tables, or a stash, all costing more RAM than this design has. Double hashing comfortably beats linear probing's worst case (36 vs. 175 probes) and reliably inserts every entry, which is why it – not cuckoo hashing – is the shipped collision strategy. This matches the qualitative README account ("cuckoo hashing was not used due to its increased memory consumption") with reproducible numbers behind it.

> These two scripts and their `results/` are untracked in git as of this writing (see **04 – Project Status & Next Steps**). They depend on `full_hash_eval.py` (already tracked) and `matplotlib`.

Name index: sorted array vs. AVL tree

Before `sorted_array.c` was implemented, `source/benchmark/name_index/` benchmarked a sorted array against an AVL tree for the RAM-only alphabetical contact index – both compiled with the

same capacity (`NI_MAX_CONTACTS = 10001`) and measured on host and on-target (`-Os` and `-O0`), across random/ascending/descending/prefix key distributions and D-cache on/off. `RESULTS.md` is measurements only and makes no recommendation on its own, but per the commit history (`3896190`) the decision followed directly from these numbers: **the sorted array was faster at the design N and was the only structure of the two that fit the RAM budget** (`SortedEntry` is 12 B vs. `AvlNode`'s 18 B, and the array needs no per-node pointers).

Database API benchmarks

`source/benchmark/database/` times every public `database_*` call (contacts, messages/chats, calls, init/recovery) on both host (`-O3`, CPU-cost sanity check only) and target (`-Os`, authoritative), sweeping contact counts from 100 up to 9,999. Method: DWT cycle counter, I-cache on / D-cache off (firmware default), 100 timed target repetitions after 10 warm-up, sector read/write counts via a storage-wrapping shim. Full tables, methodology and caveats are in `source/benchmark/database/RESULTS.md`; a self-contained HTML report with plots is built via `report/build.sh` -> `report/dbms-benchmark.html`.

Run	Status (2026-10-20)
contacts – host / target	done (4 s / 9 min)
messages / chats – host / target	done (5 s / 12 min)
calls – host / target	done (1 s / 9 min)
init / recovery – host / target	done (1 min / 28 min)

Reproducing a run

```
# pythonSim design studies (no hardware needed)
cd pythonSim && python3 hash_selection_eval.py && python3
collision_selection_eval.py

# Database API benchmark, host side (fast sanity check)
cd source/benchmark/database && scripts/run_host.sh contacts

# Database API benchmark, on target (NUCLEO + SD card over ST-LINK)
scripts/run_target.sh contacts Release
```